

CIS530 HW3 Submission Template

1.5 Writing Shakespeare (6 points)

Shakespeare Texts

Below are the Shakespeare-like texts I generated using different n-gram models:

n = 2

“Firse. Thumpees;
And but call, wrown thispe--annest:
SIROMPET:
Ander knot the a ke, slet of you yearrock ith
I come. HENRY By ther himperd. Youl thear?

BROSPULEN Keend selishe paimle sle
th note ch o'er yourall, 't, yet to thout shadly a do my ass as”

n = 3

“First mock
That not adver Prous shephew'd; an away withee were me a veing! Is demend
That it?

CLEONA:
And sength me of fould have your chan what walking his,
And is thing he bothee:' sportuner app'd senge;
And as wron, geniece forge or this buile wi”

n = 4

“First seems, some child holy officer.
Lavinia,
Doing: you,
much peace, get his chirping
A mark'd,
Chamber
Think my tears!
I have of myself, true,
And long.

Secondemn'd, about.

YOUNG SIWARD IV:

My will give't horribly girl, and malice be wine moreov"

n = 7

"First Clown:

Good Tom Drum, lend me like me--yet long'd toys

Of feather in her your letters-patents give crown does now?

CHRISTOPHER:

Sir, we hold, there cut in the punish'd of alacrity

I find the threw his gait;

He says is riggish.

MARK ANTONY:

Fu"

n = 12

"First Citizen:

That can we not; but he that will do the deed:

Or slunk not Saturnine, interrupter of the good

That noble-minded Talbot:

Never to England's chair, where he is falsely set;

One that hath no mean dependance

Upon our joint and several dig"

Discussion

As the value of n increases in the n -gram models, the quality of the generated Shakespeare-like text noticeably improves. With $n=2$, the output is almost completely nonsensical: it contains invented words, broken syntax, and frequent non-standard punctuation (e.g., "wrown thispe--annest"), indicating very short context windows that don't capture even basic English morphology. At $n=3$, the model starts generating more word-like tokens and occasional recognizable phrases, but the grammar and semantics remain inconsistent.

By $n=4$, we begin to see more coherent phrases, such as "some child holy officer" and "Think my tears!" indicating the model is capturing syntactic structure and phrase boundaries better. With $n=7$, the text flows even more naturally: it includes dialogue labels (e.g., "First Clown", "CHRISTOPHER"), stage direction-like phrases, and reasonably structured sentences, although they may still be semantically odd. At $n=12$, the output closely resembles real Shakespearean language in both form and content since there are full, grammatically valid lines, speaker attributions, and thematic elements (e.g., "noble-minded Talbot", "false set") that evoke genuine Shakespearean tone.

The reason the first word often starts with the same letter (usually “F”) is due to the consistent start padding (e.g., '~' * n) used to initialize generation, which makes the model highly likely to start with the same character (frequently 'F' from “First”). This deterministic bias is especially strong in low-n models. Overall, higher-order n-grams are able to better mimic Shakespearean elements like rhythm, repetition, archaic vocabulary, and the formatting of plays (with speaker names and stage cues).

2.2 Perplexity (6 points)

Procedure

1. I updated an N-gram model with Shakespeare's plays.
2. I calculated the perplexity for a different text: [name of non-Shakespeare text corpus]
3. I calculated the perplexity for a similar text: [name of Shakespeare-like text corpus]
4. Below are my results and analysis.

Results

Text Type	n	k	Perplexity
nytimes_article.txt	2	1	13.2533
nytimes_article.txt	3	1	13.5968
nytimes_article.txt	4	1	14.0509
shakespeare_sonnets.txt	2	1	9.6035
shakespeare_sonnets.txt	3	1	9.8677
shakespeare_sonnets.txt	4	1	10.1392

Discussion

The perplexity results clearly show that the Shakespeare-trained language model assigns **lower perplexity** to the *shakespeare_sonnets.txt* (similar text) than to the *nytimes_article.txt* (different text) across all values of **n**. For example, at **n = 2**, the perplexity is **9.60** for the sonnets versus **13.25** for the New York Times article. This gap persists and even widens slightly as **n** increases, confirming that the model is more confident and predictive on texts similar to its training data.

The reason the sonnets have lower perplexity is because they **share stylistic, syntactic, and lexical patterns** with the Shakespeare plays used to train the model. These shared features allow the n-gram model to make more accurate predictions about upcoming characters based on learned contexts. In contrast, the New York Times article represents a different genre, vocabulary, and structure, making predictions less certain and increasing perplexity.

Overall, these results reinforce that **perplexity is a reliable indicator of how well a language model fits a given text**. Lower perplexity signals stronger alignment between the test data and the training corpus, while higher perplexity reflects unfamiliarity and reduced predictive performance.

2.3 Smoothing and Interpolation (6 points)

Procedure

1. I updated an N-gram model with [text corpus used for training].
2. I calculated perplexities for [text corpus used for testing] with different values of k.
3. I calculated perplexities for [text corpus used for testing] with different combinations of lambda.
4. Below are my results and analysis.

*** When running your experiments, only change one variable at a time***

*** Also, make sure you train on the same text corpus for all experiments ***

Results for Different k Values

Text Type	n	k	Uninterpolated Perplexity	Interpolated Perplexity
shakespeare_sonnets.txt	2	0.1	4.6227	5.9427
shakespeare_sonnets.txt	2	0.5	7.8257	8.9419
shakespeare_sonnets.txt	2	1.0	9.6035	10.4399
shakespeare_sonnets.txt	2	2.0	11.1913	11.7162
shakespeare_sonnets.txt	2	3.0	11.9328	12.3048

Results for Different Lambda Combinations

Text Type	λ	Perplexity
shakespeare_sonnets.txt	[0.25, 0.25, 0.25, 0.25]	10.2594
shakespeare_sonnets.txt	[0.4, 0.3, 0.2, 0.1]	10.6377
shakespeare_sonnets.txt	[0.1, 0.2, 0.3, 0.4]	9.9419

Discussion

The results clearly show that as the smoothing parameter **k** increases, **perplexity also increases** for both interpolated and uninterpolated models. This trend indicates that excessive smoothing leads to overgeneralization—diluting the influence of meaningful character sequences from the training data. The **optimal k value is 0.1**, which yields the **lowest perplexities** (4.62 un-interpolated, 5.94 interpolated), suggesting that minimal smoothing provides the best balance between accounting for unseen n-grams and preserving the richness of learned patterns.

Regarding interpolation, different **lambda combinations** influence model performance significantly. The best-performing lambda setting is **[0.1, 0.2, 0.3, 0.4]**, which assigns higher weight to lower-order models. This configuration achieves the **lowest perplexity of 9.94**, suggesting that in this particular dataset, backing off to lower-order n-grams is beneficial—possibly due to sparsity or stylistic regularity in the sonnet corpus. In contrast, the uniform weighting ([0.25, 0.25, 0.25, 0.25]) and high-weight-on-higher-order ([0.4, 0.3, 0.2, 0.1]) configurations perform slightly worse, likely because they assume more reliable context than the data supports.

Overall, these results highlight the importance of carefully tuning **both smoothing and interpolation weights** to match the structure and size of the dataset. Models that overemphasize high-order dependencies or apply excessive smoothing tend to underperform when data is sparse or stylized, as is common in literary corpora like Shakespeare's sonnets.

3.1 Own Model's Performance (5 points)

- Validation Accuracy: 0.6700

3.2 Model Description and Analysis (15 points)

Final Model Parameters

My best model uses the following parameters:

- **n**: 4
- **k**: 1
- **lambda**: [0.5, 0.1, 0.1, 0.2, 0.1]

This combination achieved validation accuracies of 67%

Experiment Process and Final Decision

I evaluated several combinations of hyperparameters to determine the best-performing language model. My experiments included:

- Varying the order of the n-gram ($n = 2, 3, 4$)
- Testing multiple smoothing values ($k = 0.1, 1.0$)
- Trying different interpolation weights (λ values)

The model was trained per country using a character-level **NgramModelWithInterpolation**, and predictions were made by choosing the country model with the lowest perplexity for a given city name. The goal was to identify the configuration that generalizes best to new city names in the validation set.

Here is the table summarizing all trials:

n	k	λ (lambda)	Validation Accuracy
2	1	[0.5, 0.1, 0.2, 0.2]	62.37%
3	1	[0.5, 0.1, 0.2, 0.2]	66.67%
4	1	[0.5, 0.1, 0.1, 0.2, 0.1]	67.02% (Final)
5	1	[0.5, 0.1, 0.1, 0.1, 0.1, 0.1]	66.44%

The best-performing setup was a **3-gram model** with $k=1$ and $\lambda = [0.5, 0.1, 0.1, 0.2, 0.1]$. This configuration prioritized higher-order character sequences, while still incorporating some backup from lower-order models. Models with $n=2$ or $n=3$ were not expressive enough, and $n=5$ provided no additional improvement but increased complexity.

Error Analysis

Below are examples of cities my model misclassified:

```

      City Name  True Label  Predicted Label
0      bonabdan          ir          za
1         lord          ir          de
2      karrapu          ir          af
3    lysohirka          ir          fi
4    kar kande          ir          af
..      ...
292    shah mir          pk          ir
293  nazar garhi          pk          za
294    montgomery          pk          fr
295 bar bamakhel          pk          af
296      shamal          pk          af

```

[297 rows x 3 columns]

City Name	True Label	Predicted Label
bonabdan	ir	za
lord	ir	de
karrapu	ir	af
lysohirka	ir	fi
kar kande	ir	af
shah mir	pk	ir
nazar garhi	pk	za
montgomery	pk	fr
bar bamakhel	pk	af
shamal	pk	af

Error Discussion

These misclassifications reveal two key issues:

1. **Linguistic overlap:** City names like *frankfurt* and *stuttgart* contain character sequences that may overlap with other countries (e.g., *fr* in “France” or *rt* in *beijing*), leading to confusion.

2. **Limited training data:** The German dataset might lack diverse enough examples to properly weight distinctive character sequences. In contrast, Afghanistan and China examples may contain broader character distributions due to more common letter combinations.

Additionally, since this is a **character-level model**, it lacks awareness of full morphemes or semantic context. That makes it vulnerable to overfitting on frequent character trigrams that appear across multiple languages.